

✓ Day 18: Deep Dive into TensorFlow Optimizers

1. Introduction to Optimizers

Optimizers adjust the weights of your model during training to minimize the loss function. There are various optimizers, each with a different approach to updating the weights

✓ 2. Gradient Descent Optimizer

Gradient Descent is the most basic optimizer, where weights are updated by computing the gradient of the loss with respect to each weight.

```
import tensorflow as tf

# Creating a simple model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(1, activation='linear')
])

# Compiling the model with Gradient Descent optimizer
model.compile(optimizer='SGD', loss='mean_squared_error')

# Viewing the model summary
model.summary()
```

 /usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	50
dense_1 (Dense)	(None, 1)	11

Total params: 61 (244.00 B)
Trainable params: 61 (244.00 B)
Non-trainable params: 0 (0.00 B)

✓ 3. Adam Optimizer

The Adam optimizer is one of the most commonly used optimizers due to its adaptive learning rate and fast convergence.

```
import numpy as np
import tensorflow as tf

# Generating a simple dataset
```

```
X_train = np.random.rand(100, 4) # 100 samples, 4 features
y_train = np.random.rand(100, 1) # 100 samples, 1 target variable

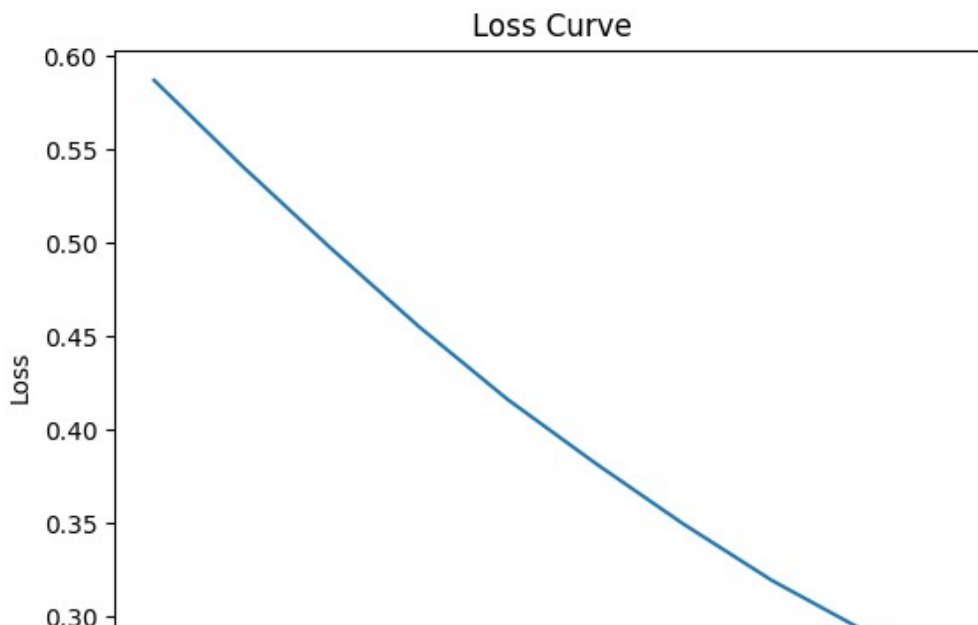
# Creating a simple model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(1, activation='linear')
])

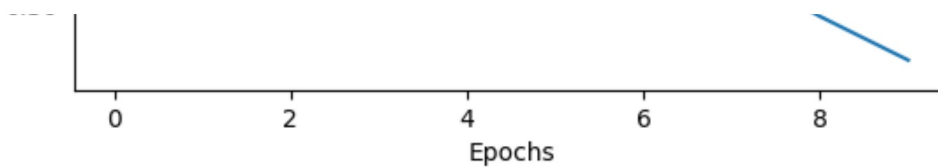
# Compiling the model with Adam optimizer
model.compile(optimizer='adam', loss='mean_squared_error')

# Training the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32)

# Plotting the loss curve
import matplotlib.pyplot as plt
plt.plot(history.history['loss'])
plt.title('Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.show()
```

Epoch 1/10
4/4 ————— 2s 10ms/step - loss: 0.5823
Epoch 2/10
4/4 ————— 0s 6ms/step - loss: 0.5503
Epoch 3/10
4/4 ————— 0s 5ms/step - loss: 0.5020
Epoch 4/10
4/4 ————— 0s 7ms/step - loss: 0.4604
Epoch 5/10
4/4 ————— 0s 10ms/step - loss: 0.4108
Epoch 6/10
4/4 ————— 0s 9ms/step - loss: 0.4004
Epoch 7/10
4/4 ————— 0s 11ms/step - loss: 0.3637
Epoch 8/10
4/4 ————— 0s 4ms/step - loss: 0.3018
Epoch 9/10
4/4 ————— 0s 4ms/step - loss: 0.3084
Epoch 10/10
4/4 ————— 0s 16ms/step - loss: 0.2734





✓ 4. RMSprop Optimizer

The RMSprop optimizer is particularly useful for models with a lot of variation in gradients, as it adjusts the learning rate based on the recent magnitudes of gradients.

```
# Compiling the model with RMSprop optimizer
model.compile(optimizer='rmsprop', loss='mean_squared_error')

# Training the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32)
```

```
Epoch 1/10
4/4 ————— 2s 6ms/step - loss: 0.2381
Epoch 2/10
4/4 ————— 0s 6ms/step - loss: 0.1897
Epoch 3/10
4/4 ————— 0s 5ms/step - loss: 0.1758
Epoch 4/10
4/4 ————— 0s 4ms/step - loss: 0.1558
Epoch 5/10
4/4 ————— 0s 4ms/step - loss: 0.1546
Epoch 6/10
4/4 ————— 0s 4ms/step - loss: 0.1540
Epoch 7/10
4/4 ————— 0s 4ms/step - loss: 0.1367
Epoch 8/10
4/4 ————— 0s 4ms/step - loss: 0.1378
Epoch 9/10
4/4 ————— 0s 4ms/step - loss: 0.1195
Epoch 10/10
4/4 ————— 0s 4ms/step - loss: 0.1091
```

✓ 5. Comparing Optimizers

Let's compare how different optimizers affect model training in terms of speed, accuracy, and loss.

Start coding or [generate](#) with AI.

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Generating a simple dataset
X_train = np.random.rand(100, 4) # 100 samples, 4 features
y_train = np.random.rand(100, 1) # 100 samples, 1 target variable

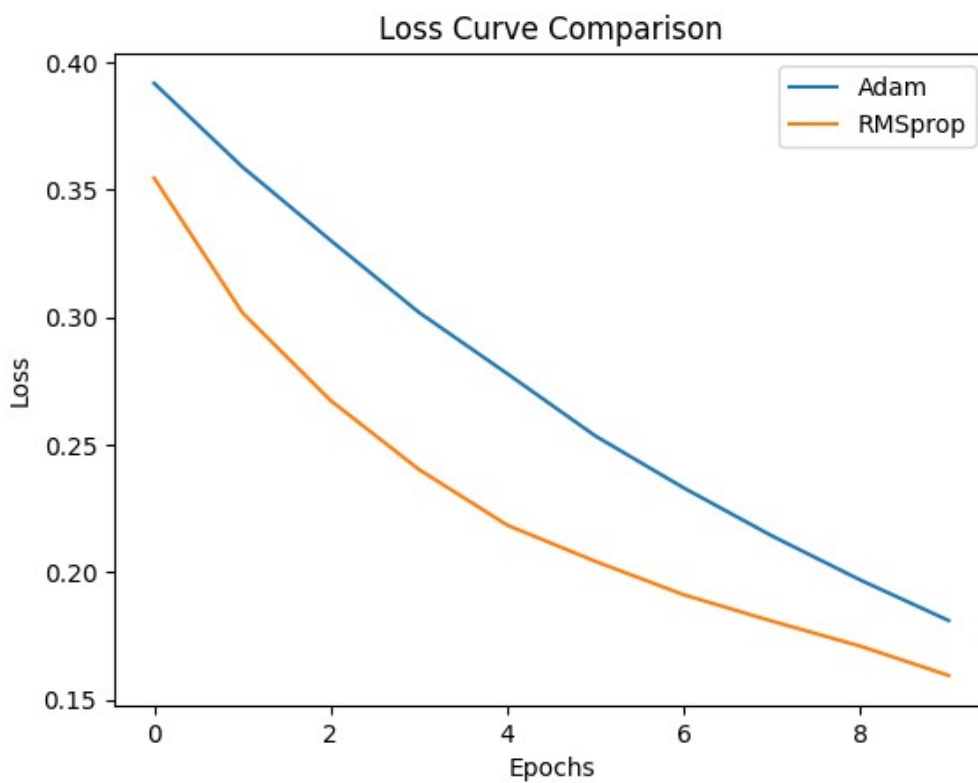
# Function to build a model
def build_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(10, activation='relu', input_shape=(4,)),
```

```
tf.keras.layers.Dense(1, activation='linear')
])
return model

# Training with Adam optimizer
model_adam = build_model()
model_adam.compile(optimizer='adam', loss='mean_squared_error')
history_adam = model_adam.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

# Training with RMSprop optimizer
model_rmsprop = build_model()
model_rmsprop.compile(optimizer='rmsprop', loss='mean_squared_error')
history_rmsprop = model_rmsprop.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

# Plotting the loss curves for Adam and RMSprop
plt.plot(history_adam.history['loss'], label='Adam')
plt.plot(history_rmsprop.history['loss'], label='RMSprop')
plt.title('Loss Curve Comparison')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Generating a simple dataset
X_train = np.random.rand(100, 4) # 100 samples, 4 features
y_train = np.random.rand(100, 1) # 100 samples, 1 target variable

# Function to build a model
def build_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(10, activation='relu', input_shape=(4,)),
```

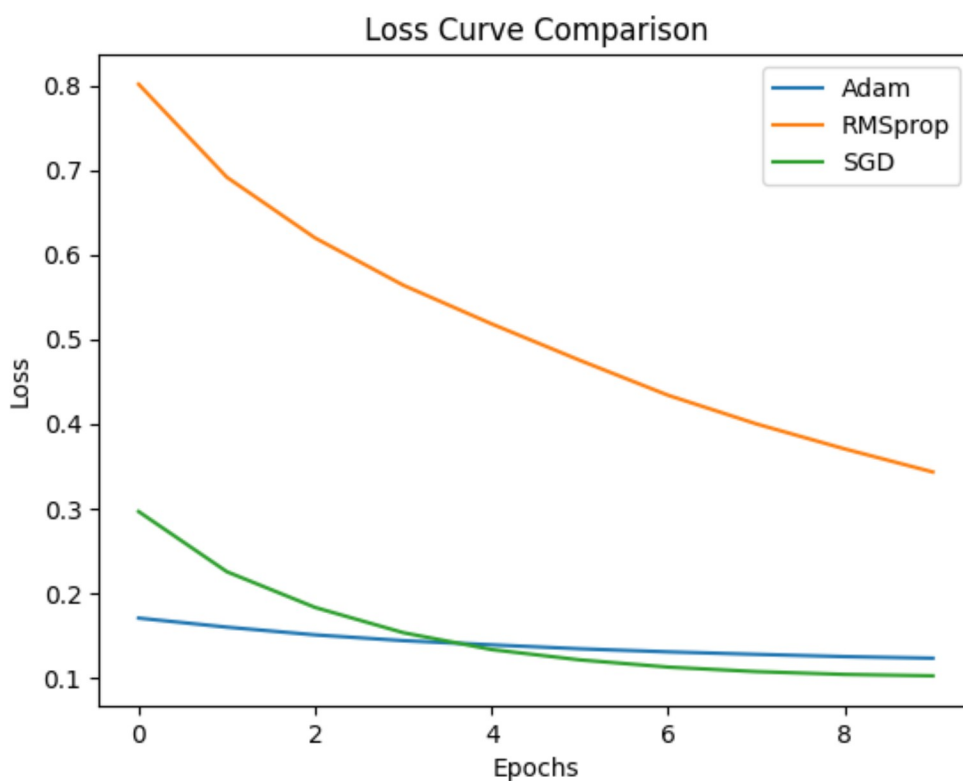
```
tf.keras.layers.Dense(1, activation='linear')
])
return model

# Training with Adam optimizer
model_adam = build_model()
model_adam.compile(optimizer='adam', loss='mean_squared_error')
history_adam = model_adam.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

# Training with RMSprop optimizer
model_rmsprop = build_model()
model_rmsprop.compile(optimizer='rmsprop', loss='mean_squared_error')
history_rmsprop = model_rmsprop.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

# Training with SGD optimizer
model_sgd = build_model()
model_sgd.compile(optimizer='sgd', loss='mean_squared_error')
history_sgd = model_sgd.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

# Plotting the loss curves for Adam, RMSprop, and SGD
plt.plot(history_adam.history['loss'], label='Adam')
plt.plot(history_rmsprop.history['loss'], label='RMSprop')
plt.plot(history_sgd.history['loss'], label='SGD')
plt.title('Loss Curve Comparison')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



Start coding or [generate](#) with AI.

